



ບົດທີ 4

ວິສະວະກຳຄວາມຕ້ອງການ

Requirement Engineering

ເນື້ອໃນ

- 1 ປະເພດຂອງຄວາມຕ້ອງການ (Functional vs. Non-functional)
- 2 ຂະບວນການວິສະວະກຳຄວາມຕ້ອງການ
- 3 ເທັກນິກການລວບລວມຄວາມຕ້ອງການ
- 4 ການຂຽນ User Stories ແລະ ເງື່ອນໄຂການຍອມຮັບ

ຄໍາແນະນຳ



ຄວາມຕ້ອງການແມ່ນ **ວັດຖຸດິບທີ່ສໍາຄັນທີ່ສຸດ** ຂອງຂະບວນການພັດທະນາຊອບແວ



ການກໍານົດຄວາມຕ້ອງການທີ່ຜິດພາດ ເປັນ **ສາເຫດອັນດັບໜຶ່ງ** ທີ່ເຮັດໃຫ້ໂຄງການຊອບແວລົ້ມເຫຼວ



ວິສະວະກຳຄວາມຕ້ອງການ ແມ່ນຂະບວນການທີ່ຊ່ວຍໃຫ້ພວກເຮົາ **ຄົ້ນຫາ, ວິເຄາະ, ລະບຸ/ກໍານົດ ແລະ ຈັດການ** ຄວາມຕ້ອງການຢ່າງມີປະສິດທິຜົນ



ຄວາມຕ້ອງການທີ່ຖືກຕ້ອງ ແມ່ນພື້ນຖານຂອງຊອບແວທີ່ປະສົບຜົນສໍາເລັດ

ປະເພດຂອງຄວາມຕ້ອງການ



ຄວາມຕ້ອງການທີ່ເປັນໜ້າທີ່

ລະບຸວ່າ "ລະບົບຕ້ອງເຮັດຫຍັງໄດ້ແດ່" (What the system must do)

ເປັນໜ້າທີ່ (Functions), ລັກສະນະການໃຊ້ງານ (Features), ຫຼື ພຶດຕິກຳສະເພາະຂອງລະບົບ

ຕົວຢ່າງ: "ຜູ້ໃຊ້ຕ້ອງສາມາດ ຄົ້ນຫາ ປຶ້ມດ້ວຍຊື່ປຶ້ມ ຫຼື ຊື່ຜູ້ແຕ່ງໄດ້."



ຄວາມຕ້ອງການທີ່ບໍ່ເປັນໜ້າທີ່

ກຳນົດວ່າ "ລະບົບຄວນຈະເປັນແນວໃດ" (How the system should be)

ການກຳນົດ ຄຸນລັກສະນະ, ຄຸນນະພາບ, ມາດຕະຖານ, ຫຼື ຂໍ້ຈຳກັດ ຂອງລະບົບ

ຕົວຢ່າງ: "ໜ້າເວັບຕ້ອງໂຫຼດສຳເລັດພາຍໃນ 3 ວິນາທີ."

! ຄວາມສຳຄັນຂອງການແຍກປະເພດຄວາມຕ້ອງການ



ມີຜົນຕໍ່ການອອກແບບສະຖາປັດຕະຍະກຳ



ມີຜົນຕໍ່ການວາງແຜນ ແລະ ການປະເມີນລາຄາ



ມີຜົນຕໍ່ການທົດສອບ

ຄວາມຕ້ອງການທີ່ເປັນໜ້າທີ່ (Functional Requirements)



ຄໍານິຍາມ

ຄືຄວາມຕ້ອງການທີ່ລະບຸວ່າ "ລະບົບຕ້ອງເຮັດຫຍັງໄດ້ແດ່" (What the system must do)

ມັນຄືໜ້າທີ່ (Functions), ລັກສະນະການໃຊ້ງານ (Features), ຫຼື ພຶດຕິກຳສະເພາະຂອງລະບົບ



ລັກສະນະ



ມັກຈະຖືກອະທິບາຍດ້ວຍ ຄໍາກຳມະ (Verb) ແລະ ຜົນໄດ້ຮັບຂອງການກະທຳ



ມັກຂຽນໃນຮູບແບບ "ຜູ້ໃຊ້ຕ້ອງສາມາດ [ເຮັດບາງຢ່າງ] ໄດ້"



ລະບຸການກະທຳ ແລະ ກົດທາງທຸລະກິດທີ່ລະບົບຕ້ອງປະຕິບັດ



ຕົວຢ່າງ



"ຜູ້ໃຊ້ຕ້ອງສາມາດ ຄົ້ນຫາ ປຶ້ມດ້ວຍຊື່ປຶ້ມ ຫຼື ຊື່ຜູ້ແຕ່ງໄດ້."



"ລະບົບຕ້ອງສາມາດໃຫ້ສະມາຊິກ ຍິນ ປຶ້ມໄດ້ສູງສຸດ 5 ຫົວຕໍ່ຄັ້ງ."



"ລະບົບຕ້ອງ ສົ່ງ ອີເມລແຈ້ງເຕືອນ ອັດຕະໂນມັດ 2 ມື້ ກ່ອນຈະຮອດກຳນົດສິ່ງປຶ້ມຄືນ."



"ບັນນາຮັກຕ້ອງສາມາດ ເພີ່ມ ຂໍ້ມູນປຶ້ມໃໝ່ເຂົ້າໃນລະບົບໄດ້."

ຄວາມຕ້ອງການທີ່ບໍ່ເປັນໜ້າທີ່ (Non-functional Requirements)



ຄໍານິຍາມ

ຄືຄວາມຕ້ອງການທີ່ບໍ່ໄດ້ບອກວ່າລະບົບເຮັດຫຍັງ, ແຕ່ກໍານົດວ່າ "ລະບົບຄວນຈະເປັນແນວໃດ" (How the system should be)

ມັນຄືການກໍານົດ ຄຸນລັກສະນະ, ຄຸນນະພາບ, ມາດຕະຖານ, ຫຼື ຂໍ້ຈຳກັດ ຂອງລະບົບ



ລັກສະນະ



ມັກຈະເປັນສິ່ງທີ່ "ວັດແທກໄດ້" (Measurable)



ການເວົ້າວ່າ "ລະບົບຕ້ອງໄວ" ບໍ່ແມ່ນ NFR ທີ່ດີ



ການເວົ້າວ່າ "ລະບົບຕ້ອງສະແດງຜົນການຄົ້ນຫາພາຍໃນ 2 ວິນາທີ" ເປັນ NFR ທີ່ດີ



ປະເພດ ແລະ ຕົວຢ່າງ



ປະສິດທິພາບ (Performance): "ໜ້າເວັບຕ້ອງໂຫຼດສໍາເລັດພາຍໃນ 3 ວິນາທີ."



ຄວາມປອດໄພ (Security): "ລະຫັດຜ່ານຂອງຜູ້ໃຊ້ທຸກຄົນຕ້ອງຖືກເຂົ້າລະຫັດ (encrypted) ໃນຖານຂໍ້ມູນ."



ຄວາມໜ້າເຊື່ອຖື (Reliability): "ລະບົບຕ້ອງສາມາດໃຊ້ງານໄດ້ 99.9% ຂອງເວລາ (Uptime)."



ຄວາມງ່າຍໃນການໃຊ້ງານ (Usability): "ຜູ້ໃຊ້ໃໝ່ຕ້ອງສາມາດຮຽນຮູ້ວິທີໃຊ້ໄດ້ພາຍໃນ 5 ນາທີ ໂດຍບໍ່ຕ້ອງອ່ານຄູ່ມື."

ຂະບວນການວິສະວະກຳຄວາມຕ້ອງການ

ວິສະວະກຳຄວາມຕ້ອງການບໍ່ແມ່ນກິດຈະກຳທີ່ເຮັດພຽງຄັ້ງດຽວແລ້ວຈົບ, ແຕ່ເປັນຂະບວນການທີ່ເຮັດຊ້ຳ ແລະ ຕໍ່ເນື່ອງຕະຫຼອດໂຄງການ

໒ ຂະບວນການທີ່ເຮັດຊ້ຳ

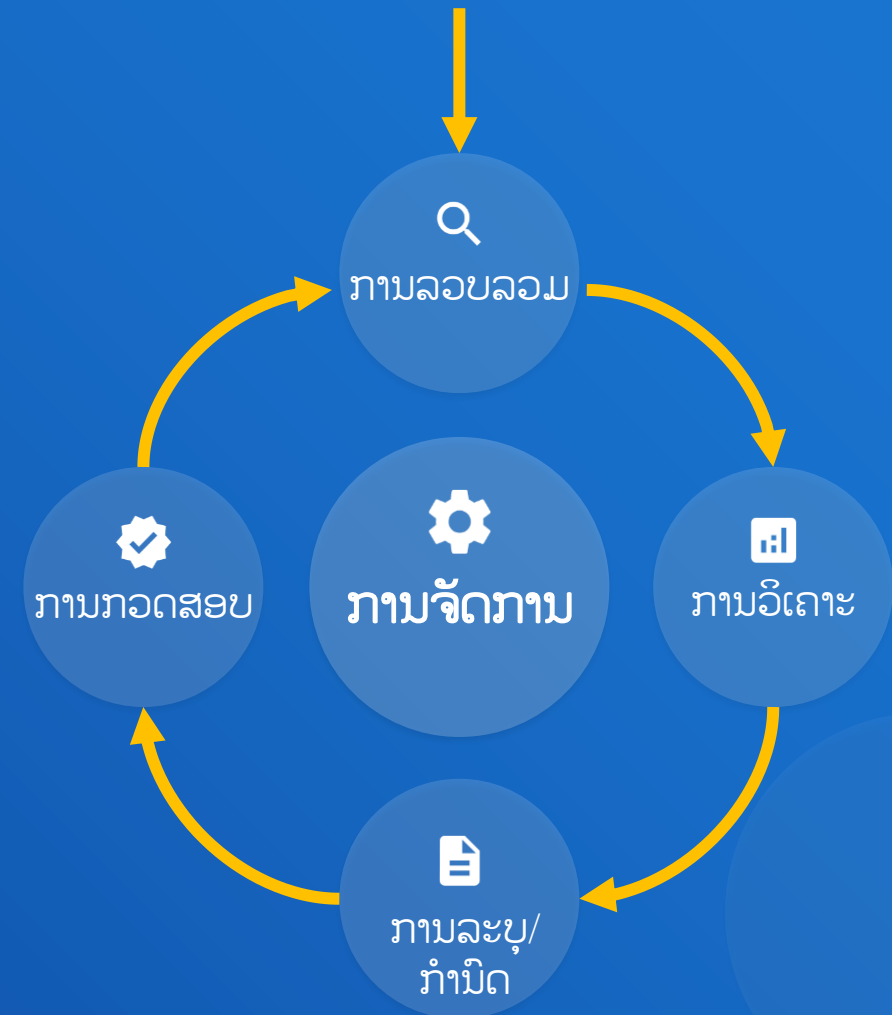
ທຸກຂັ້ນຕອນມີຄວາມສຳພັນກັນ ແລະ ຕ້ອງກັບຄືນໄປແກ້ໄຂຂັ້ນຕອນກ່ອນໜ້າໜ້າ

✱ ການຈັດການເປັນແຖນກາງ

ການຈັດການຄວາມຕ້ອງການເປັນກິດຈະກຳທີ່ເຊື່ອມໂຍງກັບທຸກຂັ້ນຕອນ

~ ການປັບປຸງຕະຫຼອດເວລາ

ຂະບວນການນີ້ຊ່ວຍໃຫ້ພວກເຮົາປັບປຸງຄວາມຕ້ອງການຕະຫຼອດໂຄງການ



ການລວບລວມຄວາມຕ້ອງການ (Requirement Elicitation)

❓ ຄໍາຖາມຫຼັກ

ເຮົາຈະດຶງຄວາມຕ້ອງການອອກມາຈາກຜູ້ມີສ່ວນຮ່ວມ (Stakeholders) ໄດ້ແນວໃດ?

🧠 ຄໍາອະທິບາຍ

ນີ້ຄືຂັ້ນຕອນ "ການຄົ້ນຫາ". ຄວາມຕ້ອງການມັກຈະຖືກເຊື່ອງຊ້ອນຢູ່, ບໍ່ຊັດເຈນ, ແລະ ບາງຄັ້ງລູກຄ້າເອງກໍຍັງບໍ່ຮູ້ວ່າຕ້ອງການຫຍັງແທ້ໆ

📁 ຜົນໄດ້ຮັບ

📖 ຂໍ້ມູນດິບຈຳນວນຫຼາຍ

👥 ບັນທຶກການປະຊຸມ

💡 ຄວາມເຂົ້າໃຈເບື້ອງຕົ້ນກ່ຽວກັບບັນຫາ

🕒 ເທັກນິກທີ່ນິຍົມໃຊ້

👤 ການສຳພາດ (Interviews)

👥 ເວີກຊັອບ (Workshops)

👁️ ການສັງເກດການ (Observation)

📄 ການສຳຫຼວດ (Questionnaires)

📁 ການວິເຄາະເອກະສານ (Document Analysis)

💡 ເຄັດລັບສໍາຄັນ

ບໍ່ມີເທັກນິກໃດທີ່ດີທີ່ສຸດພຽງອັນດຽວ, ການເລືອກໃຊ້ເຄື່ອງມືທີ່ຖືກຕ້ອງກັບສະຖານະການ ຄືສິ່ງທີ່ຍິ່ງບອກລະຫວ່າງນັກວິເຄາະລະບົບມືໃໝ່ ແລະ ມີອາຊີບ

ການວິເຄາະຄວາມຕ້ອງການ (Requirement Analysis)

❓ ຄໍາຖາມຫຼັກ

ຂໍ້ມູນທີ່ໄດ້ມາ ມີຄວາມໝາຍວ່າແນວໃດ? ມັນ
ສົມເຫດສົມຜົນ ແລະ ຄົບຖ້ວນບໍ່?

💡 ຄໍາອະທິບາຍ

ນີ້ຄືຂັ້ນຕອນ "ການຈັດລະບຽບ". ເຮົາຈະນຳເອົາ
ຂໍ້ມູນດິບທີ່ກະແຈກກະຈາຍມາຈັດກຸ່ມ, ຊອກ
ຫາຄວາມຂັດແຍ່ງ, ແລະ ສ້າງຄວາມເຂົ້າໃຈແບບ
ເລິກເຊິ່ງ



ຜົນໄດ້ຮັບ: ຄວາມຕ້ອງການທີ່ມີການຈັດ
ລະບຽບ ແລະ ຕົກລົງເຫັນດີກັນແລ້ວ

👁️ ກິດຈະກຳ



ການຈັດກຸ່ມ ແລະ ຈັດລຳດັບ
ຄວາມສຳຄັນ
ຈັດກຸ່ມຄວາມຕ້ອງການທີ່
ກ່ຽວຂ້ອງກັນ ແລະ ຮ່ວມມືກັບ
Product Owner



ການສ້າງແບບຈຳລອງ
(Modeling)
ສ້າງແຜນວາດຕ່າງໆ
ເພື່ອຊ່ວຍໃຫ້ເຫັນພາບ,
ເຊັ່ນ: Use Case
Diagram



ການເຈລະຈາ ແລະ ແກ້ໄຂຂໍ້
ຂັດແຍ່ງ
ຜູ້ມີສ່ວນຮ່ວມຕ່າງກັນ ອາດຈະ
ມີຄວາມຕ້ອງການທີ່ຂັດແຍ່ງກັນ



ການວິເຄາະຄວາມ
ຕ້ອງການທີ່ຊັບຊ້ອນ
ຊອກຫາຄວາມ
ຕ້ອງການທີ່ຊ້ຳກັນ ຫຼື
ຂັດແຍ່ງກັນ

🌟 ຕົວຢ່າງຈິງ

ຝ່າຍການເງິນຕ້ອງການລະບົບທີ່ປອດໄພທີ່ສຸດ, ແຕ່ຝ່າຍການຕະຫຼາດ
ຕ້ອງການລະບົບທີ່ໃຊ້ງ່າຍທີ່ສຸດ. ໜ້າທີ່ຂອງເຮົາຄືການຊ່ວຍຫາຈຸດທີ່ສົມດຸນ.

ການລະບຸ/ກຳນົດຄວາມຕ້ອງການ (Requirement Specification)

❓ ຄຳຖາມຫຼັກ

ເຮົາຈະບັນທຶກຄວາມຕ້ອງການທີ່ຕົກລົງກັນແລ້ວ ໃຫ້ທຸກຄົນເຂົ້າໃຈກົງກັນໄດ້ແນວໃດ?

🧠 ຄຳອະທິບາຍ

ນີ້ຄືຂັ້ນຕອນ "ການຂຽນເອກະສານ". ເຮົາຈະປ່ຽນຄວາມເຂົ້າໃຈທີ່ຢູ່ໃນຫົວ ໃຫ້ກາຍເປັນເອກະສານທີ່ຊັດເຈນ, ສາມາດວັດແທກໄດ້, ແລະ ບໍ່ສາມາດຕີຄວາມໝາຍເປັນຢ່າງອື່ນໄດ້

✅ ຜົນໄດ້ຮັບ: ເອກະສານ SRS ຫຼື Product Backlog ທີ່ພ້ອມໃຫ້ທີມພັດທະນານຳໄປໃຊ້ງານ

📄 ຮູບແບບຂອງເອກະສານ

📑 Waterfall

Software Requirement Specification (SRS) ທີ່ລະບຸທຸກຢ່າງຢ່າງລະອຽດ

🔄 Agile

User Stories ທີ່ຂຽນໃສ່ໃນ Product Backlog ແບບງ່າຍດາຍ

🔍 ຄຸນລັກສະນະຂອງເອກະສານທີ່ດີ

✅ ຄວາມຄົບຖ້ວນ

🎯 ຊັດເຈນ

👉 ບໍ່ຂັດແຍ່ງກັນເອງ

⚖️ ສາມາດທົດສອບໄດ້

ການກວດສອບຄວາມຕ້ອງການ (Requirement Validation)

❓ ຄໍາຖາມຫຼັກ

ເອກະສານທີ່ເຮົາຂຽນຂຶ້ນມາ, ມັນແມ່ນສິ່ງທີ່ລູກຄ້າຕ້ອງການແທ້ໆ ບໍ?

🧠 ຄໍາອະທິບາຍ

ນີ້ຄືຂັ້ນຕອນ "ການຢືນຢັນ". ເປັນການກວດສອບວ່າສິ່ງທີ່ເຮົາ "ເຂົ້າໃຈ" ແລະ "ບັນທຶກ" ໄວ້, ມັນກົງກັບສິ່ງທີ່ລູກຄ້າ "ຕ້ອງການ" ແທ້ໆ



ຜົນໄດ້ຮັບ: ຄວາມໝັ້ນໃຈວ່າເຮົາກໍາລັງຈະສ້າງສິ່ງທີ່ຖືກຕ້ອງ

🕒 ເທັກນິກທີ່ນິຍົມໃຊ້



ການທົບທວນ

ການນໍາເອົາເອກະສານ SRS ຫຼື User Stories ໄປໃຫ້ລູກຄ້າ ແລະ ຜູ້ມີສ່ວນຮ່ວມອ່ານ ແລະ ຢືນຢັນຄວາມຖືກຕ້ອງ



ການສ້າງຈໍາລອງ

ສ້າງແບບຈໍາລອງທີ່ກົດໄດ້ (Mock-up) ໄປໃຫ້ລູກຄ້າລອງໃຊ້ ເພື່ອເກັບກໍາຄໍາຄິດເຫັນ

🌟 ປະໂຫຍດຂອງການກວດສອບ



ຫຼຸດຄວາມຜິດພາດໃນຂັ້ນຕອນຕົ້ນ



ເພີ່ມຄວາມເຂົ້າໃຈຮ່ວມກັນລະຫວ່າງທີມ



ຫຼຸດຕົ້ນທຶນ ແລະ ເວລາການພັດທະນາ

ການຈັດການຄວາມຕ້ອງການ (Requirement Management)

❓ ຄໍາຖາມຫຼັກ

ເມື່ອມີການປ່ຽນແປງເກີດຂຶ້ນ, ເຮົາຈະຈັດການມັນແນວໃດ?

🧠 ຄໍາອະທິບາຍ

ກິດຈະກຳນີ້ຈະເກີດຂຶ້ນຕະຫຼອດເວລາ. ໂລກຄວາມເປັນຈິງຄື ຄວາມຕ້ອງການຈະປ່ຽນແປງສະເໝີ. ການຈັດການຄວາມຕ້ອງການຄືຂະບວນການໃນການຄວບຄຸມການປ່ຽນແປງ



ຜົນໄດ້ຮັບ: ຄວາມສາມາດໃນການປັບຕົວເຂົ້າກັບການປ່ຽນແປງໄດ້ຢ່າງເປັນລະບົບ



ກິດຈະກຳ



ຄວບຄຸມການປ່ຽນແປງ

ສ້າງຂະບວນການທີ່ເປັນລະບົບ ເມື່ອມີຄືນຂໍປ່ຽນແປງຄວາມຕ້ອງການ



ການຕິດຕາມ

ສາມາດຕິດຕາມໄດ້ວ່າ ຄວາມຕ້ອງການແຕ່ລະຂໍ້ ເຊື່ອມໂຍງກັບສ່ວນໃດຂອງການອອກແບບ, ໂຄດ, ແລະ ການທົດສອບ

💡 ຕົວຢ່າງຈິງ

ຫຼັງຈາກພັດທະນາໄປໄດ້ 1 ເດືອນ, ຜູ້ບໍລິຫານຂໍ ປ່ຽນແປງໂດຍການເພີ່ມຄວາມຕ້ອງການ "ຕ້ອງສາມາດຈອງອຸປະກອນ (ເຊັ່ນ: ໂປຣເຈັກເຕີ) ພ້ອມກັບຫ້ອງໄດ້". ທີມງານກໍຈະນຳເອົາຄວາມຕ້ອງການໃໝ່ນີ້ເຂົ້າສູ່ຂະບວນການວິເຄາະ ແລະ ຈັດລຳດັບຄວາມສຳຄັນໃໝ່.

ເທັກນິກການລວບລວມ: ການສຳພາດ (Interviews)

👤 ຄຳນິຍາມ

ຄືການສືບທະນາແບບມີເປົ້າໝາຍ ລະຫວ່າງ **ນັກວິເຄາະ** ກັບ **ຜູ້ມີສ່ວນຮ່ວມ** ເພື່ອເກັບກຳຂໍ້ມູນ, ຄວາມຄິດເຫັນ ແລະ ຄວາມຕ້ອງການແບບເລິກເຊິ່ງ

🕒 ຄວນໃຊ້ເມື່ອໃດ

- ✓ ເມື່ອຕ້ອງການຄວາມເຂົ້າໃຈ **ເຊິ່ງເລິກໃນ** ຫົວຂໍ້ທີ່ຊັບຊ້ອນ
- ✓ ເມື່ອຕ້ອງການເຂົ້າໃຈ **ມຸມມອງ** ແລະ **ປະສົບການສ່ວນຕົວ** ຂອງຜູ້ໃຊ້
- ✓ ເມື່ອຜູ້ມີສ່ວນຮ່ວມຢູ່ **ກະແຈກກະຈາຍ** ແລະ ບໍ່ສາມາດມາປະຊຸມພ້ອມກັນໄດ້

📌 ປະເພດຂອງການສຳພາດ

☰ ມີໂຄງສ້າງ

ຖາມຕາມລາຍການທີ່ກຽມໄວ້

💬 ບໍ່ມີໂຄງສ້າງ

ເລີ່ມຕົ້ນດ້ວຍຫົວຂໍ້ກວ້າງໆ

🗨️ ເຄິ່ງໂຄງສ້າງ

ມີຄຳຖາມຫຼັກ ແລະ ເປີດໂອກາດຖາມຕື່ມອອກ

👍 ຂໍ້ດີ

- ★ ໄດ້ຂໍ້ມູນຄຸນນະພາບສູງ
- ★ ສ້າງຄວາມສຳພັນທີ່ດີ
- ★ ສາມາດຖາມຕື່ມອອກໄດ້ທັນທີ

👎 ຂໍ້ເສຍ

- 🕒 ໃຊ້ເວລາຫຼາຍ
- 👥 ອາດມີຄວາມລຳອຽງ
- 🗣️ ບໍ່ເໝາະກັບກຸ່ມ

💡 ຕົວຢ່າງ

ນັກວິເຄາະສຳພາດບັນນາຮັກ: "ຊ່ວຍອະທິບາຍຂັ້ນຕອນການໃຫ້ຢືມປຶ້ມໃນປັດຈຸບັນໃຫ້ຟັງແດ່?", "ຖ້າໃຫ້ຂໍ້ຫຍັງກໍໄດ້ໃນລະບົບໃໝ່, ຢາກໄດ້ຫຍັງຫຼາຍທີ່ສຸດ?"

ເທັກນິກການລວບລວມ: ເວີກຊັອບ (Workshops)

👥 ຄຳນິຍາມ

ຄືການປະຊຸມທີ່ມີການວາງແຜນ ແລະ ອຳນວຍຄວາມສະດວກເປັນຢ່າງດີ, ໂດຍການເຊີນກຸ່ມຜູ້ມີສ່ວນຮ່ວມທີ່ສຳຄັນ ມາລວມຕົວກັນໃນສະຖານທີ່ດຽວ ເພື່ອລະດົມສະໝອງ, ວິເຄາະ, ແລະ ຕັດສິນໃຈບັນຫາຮ່ວມກັນ

🕒 ຄວນໃຊ້ເມື່ອໃດ

- ➡ ເມື່ອຕ້ອງການຫາຂໍ້ຕົກລົງຮ່ວມກັນຈາກຫຼາຍຝ່າຍທີ່ມີຄວາມຄິດເຫັນຕ່າງກັນ
- 💡 ເມື່ອຕ້ອງການລະດົມສະໝອງຫາແນວຄວາມຄິດໃໝ່ໆ
- 🗣️ ເມື່ອຕ້ອງການເລັ່ງຂະບວນການເກັບກຳຄວາມຕ້ອງການໃຫ້ສຳເລັດໃນເວລາອັນສັ້ນ

⚙️ ອົງປະກອບສຳຄັນ

👤 ຜູ້ອຳນວຍຄວາມສະດວກ

ຄວບຄຸມການປະຊຸມໃຫ້ດຳເນີນໄປຢ່າງລຽບງ່າຍ

☞ ຜູ້ບັນທຶກ

ຈົດບັນທຶກການປະຊຸມ ແລະ ຂໍ້ຕົກລົງຕ່າງໆ

👍 ຂໍ້ດີ

- ★ ປະສິດທິພາບສູງໃນການຫາຂໍ້ຕົກລົງ
- ★ ຫຼຸດເວລາການປະສານງານ
- ★ ສ້າງຄວາມຮູ້ສຶກເປັນເຈົ້າຂອງຮ່ວມກັນ

🗨️ ຂໍ້ເສຍ

- 📅 ຈັດຕາຕະລາງໄດ້ຍາກ
- 👥 ອາດຖືກຄອບງຳໂດຍຄົນສຽງດັງ
- 📷 ມີຄ່າໃຊ້ຈ່າຍ

💡 ຕົວຢ່າງ

ທີມງານຕ້ອງການອອກແບບ Feature "ການຄົ້ນຫາປຶ້ມ" ໃນລະບົບຫ້ອງສະໝຸດ. ພວກເຂົາຈັດເວີກຊັອບ ໂດຍເຊີນ ບັນນາຮັກ, ນັກສຶກສາ, ພະນັກງານ IT, ແລະ ຫົວໜ້າທີມພັດທະນາ ເຂົ້າມາຮ່ວມກັນ. ທຸກຄົນຊ່ວຍກັນແຕ້ມຮູບຮ່າງໜ້າຕາເທິງກະດານຂາວ ແລະ ຕົກລົງກັນວ່າຄວນຈະມີ Filter ຫຍັງແດ່.

ເທັກນິກການລວບລວມ: ການສຳຫຼວດ (Surveys)

📋 ຄຳນິຍາມ

ຄືການສ້າງຊຸດຄຳຖາມທີ່ເປັນລາຍລັກ ອັກສອນ ແລ້ວສົ່ງໄປໃຫ້ກຸ່ມຄົນຈຳນວນ ຫຼາຍ ເພື່ອເກັບກຳຂໍ້ມູນທາງດ້ານ ປະລິມານ ຫຼື ຄວາມຄິດເຫັນທົ່ວໄປ

🕒 ຄວນໃຊ້ເມື່ອໃດ

- 👤 ເມື່ອຕ້ອງການຂໍ້ມູນຈາກຜູ້ໃຊ້ຈຳນວນຫຼາຍທີ່ຢູ່ກະແຈກກະຈາຍກັນ
- ⚙️ ເມື່ອຕ້ອງການຢືນຢັນສົມມຸດຕິຖານ ຫຼື ວັດແທກຄວາມພໍໃຈ
- 📊 ເມື່ອຕ້ອງການຂໍ້ມູນທາງດ້ານສະຖິຕິ

❓ ປະເພດຂອງຄຳຖາມ

✅ ຄຳຖາມປິດ

ມີຕົວເລືອກໃຫ້ຕອບ: ແມ່ນ/ບໍ່, ຫຼາຍ/ຕົວເລືອກ, ຫຼື ຄະແນນ

☰ ຄຳຖາມເປີດ

ໃຫ້ຜູ້ຕອບຂຽນອະທິບາຍຄວາມຄິດເຫັນໄດ້ຢ່າງອິດສະຫຼະ

👍 ຂໍ້ດີ

- 📝 ເຂົ້າເຖິງຄົນຈຳນວນຫຼາຍໄວ, ຕົ້ນທຶນຕ່ຳ
- 🔄 ໃຫ້ຂໍ້ມູນປະລິມານງ່າຍຕໍ່ການວິເຄາະ
- 🕒 ຜູ້ຕອບສາມາດຕອບໄດ້ຕາມເວລາທີ່ສະດວກ

🗨️ ຂໍ້ເສຍ

- 📉 ອັດຕາການຕອບກັບມັກຕ່ຳ
- 🕒 ບໍ່ສາມາດຖາມຕໍ່ຍອດໄດ້ທັນທີ
- 🧠 ບໍ່ໃຫ້ຂໍ້ມູນແບບເລິກເຊິ່ງ

💡 ຕົວຢ່າງ

ຫຼັງຈາກເປີດໃຊ້ລະບົບຫ້ອງສະໝຸດ 3 ເດືອນ, ທີມງານສ້າງແບບສຳຫຼວດອອນລາຍ (Google Forms) ແລ້ວສົ່ງໃຫ້ນັກສຶກສາທຸກຄົນ. ຄຳຖາມ: "ທ່ານພໍໃຈກັບຄວາມໄວໃນການຄົ້ນຫາຫຼາຍປານໃດ? (1-5)", "Feature ໃດທີ່ທ່ານມັກທີ່ສຸດ?"

ການຂຽນ User Stories (ໃນບໍລິບົດຂອງ Agile)

📖 User Story ແມ່ນຫຍັງ?

ຄືຄໍາອະທິບາຍສັ້ນໆ ກ່ຽວກັບ Feature ໃດໜຶ່ງ ທີ່ຂຽນຂຶ້ນຈາກ **ມຸມມອງຂອງຜູ້ໃຊ້** ບໍ່ແມ່ນການລະບຸລາຍລະອຽດທາງເຕັກນິກ, ແຕ່ເປັນການບອກເລົ່າເຖິງ:

- 👤 "ໃຜ" (Who)
- 🔧 "ຕ້ອງການຫຍັງ" (What)
- 💡 "ເພື່ອຫຍັງ" (Why)

🧠 ຈຸດປະສົງຫຼັກ

ບໍ່ແມ່ນເພື່ອເປັນເອກະສານທີ່ສົມບູນ, ແຕ່ເປັນ**"ຄໍາເຊີນຊວນໃຫ້ເລີ່ມຕົ້ນການສົນທະນາ"**

ລະຫວ່າງ Product Owner, Development Team, ແລະ Stakeholders ລາຍລະອຽດຈະຖືກຕື່ມເຕັມໃນລະຫວ່າງການສົນທະນານັ້ນ

👤 ໂຄງສ້າງຂອງ User Story

ຮູບແບບມາດຕະຖານ

ໃນຖານະ [ໃຜ], ຂ້ອຍຕ້ອງການ [ເຮັດຫຍັງ], ເພື່ອທີ່ຂ້ອຍຈະສາມາດ [ໄດ້ຮັບຜົນປະໂຫຍດຫຍັງ]

As a [Who], I want [What], so that [Why]

👤 [ໃຜ] (Who)

ລະບຸ "ບົດບາດ" ຂອງຜູ້ໃຊ້ທີ່ຕ້ອງການ Feature ນີ້

🔧 [ເຮັດຫຍັງ] (What)

ອະທິບາຍ "ການກະທຳ" ຫຼື "ໜ້າທີ່" ທີ່ຜູ້ໃຊ້ຢາກເຮັດ

💡 [ເພື່ອຫຍັງ] (Why)

ອະທິບາຍ "ເຫດຜົນ" ຫຼື "ຄຸນຄ່າ" ທີ່ຜູ້ໃຊ້ຈະໄດ້ຮັບ

❌ User Story ທີ່ບໍ່ດີ

"ຜູ້ໃຊ້ຕ້ອງສາມາດຄົ້ນຫາໄດ້."
ບໍ່ຮູ້ວ່າໃຜ, ຄົ້ນຫາຫຍັງ, ເພື່ອຫຍັງ

✅ User Story ທີ່ດີ

"ໃນຖານະສະມາຊິກທ້ອງສະໝຸດ, ຂ້ອຍຕ້ອງການຄົ້ນຫາປຶ້ມດ້ວຍຊື່ຜູ້ແຕ່ງ, ເພື່ອທີ່ຂ້ອຍຈະສາມາດຊອກຫາປຶ້ມທັງໝົດຂອງຜູ້ແຕ່ງທີ່ຂ້ອຍມັກໄດ້ຢ່າງໄວ."

ຄຸນລັກສະນະຂອງ User Story ທີ່ດີ (The "INVEST" Model)



ຫຼັກການ INVEST

ເພື່ອໃຫ້ແນ່ໃຈວ່າ User Story ມີຄຸນນະພາບ, ເຮົາມັກຈະໃຊ້ຫຼັກການ "INVEST"

I Independent

ເປັນອິດສະຫຼະຈາກກັນ, ສາມາດຈັດລຳດັບ ແລະ ພັດທະນາໄດ້ໂດຍບໍ່ຕິດຂັດ

N Negotiable

ບໍ່ແມ່ນສັນຍາທີ່ຕາຍຕົວ, ເປັນຈຸດເລີ່ມຕົ້ນຂອງການສົນທະນາ

V Valuable

ສ້າງຄຸນຄ່າທີ່ຊັດເຈນໃຫ້ກັບຜູ້ໃຊ້ ຫຼື ລູກຄ້າ

E Estimable

ທີມພັດທະນາຕ້ອງສາມາດປະເມີນ "ຂະໜາດ" ຫຼື "ຄວາມພະຍາຍາມ" ທີ່ຕ້ອງໃຊ້

S Small

ມີຂະໜາດນ້ອຍພໍທີ່ຈະເຮັດໃຫ້ສຳເລັດໄດ້ພາຍໃນ Sprint ດຽວ

T Testable

ຕ້ອງມີວິທີທີ່ຊັດເຈນໃນການກວດສອບວ່າ Story ເຮັດສຳເລັດ ແລະ ຖືກຕ້ອງ



ຄວາມສຳຄັນຂອງ INVEST

ຄຸນລັກສະນະທີ່ສຳຄັນທີ່ສຸດ

- ! **Testable** ແລະ **Valuable** ແມ່ນສອງສ່ວນທີ່ສຳຄັນທີ່ສຸດ
- ! **Small** ແລະ **Estimable** ຊ່ວຍໃຫ້ທີມສາມາດວາງແຜນການພັດທະນາໄດ້ຢ່າງມີປະສິດທິຜົນ
- ! **Independent** ແລະ **Negotiable** ຊ່ວຍໃຫ້ມີຄວາມຍືດຫຍຸ້ນໃນການຈັດການ Sprint

ບົດສະຫຼຸບ

User Story ແລະ Acceptance Criteria ຄືຄູ່ຮ່ວມມືທີ່ຂາດກັນບໍ່ໄດ້ໃນການພັດທະນາແບບ Agile. User Story ບອກເຮົາວ່າ "ຈະສ້າງຫຍັງ ເພື່ອໃຜ ແລະ ເພື່ອຫຍັງ", ໃນຂະນະທີ່ Acceptance Criteria ບອກເຮົາວ່າ "ເຮົາຈະຮູ້ໄດ້ແນວໃດວ່າສິ່ງທີ່ສ້າງນັ້ນ ສຳເລັດ ແລະ ຖືກຕ້ອງແລ້ວ".

ເງື່ອນໄຂການຍອມຮັບ (Acceptance Criteria)

✖ ຄຳນິຍາມ

Acceptance Criteria (AC) ຄືຊຸດຂອງເງື່ອນໄຂທີ່ລະບຸໄວ້ຢ່າງຊັດເຈນ, ສາມາດທົດສອບໄດ້, ເຊິ່ງຕ້ອງເປັນຈິງທັງໝົດເພື່ອໃຫ້ User Story ໃດໜຶ່ງ ຖືກຖືວ່າ "ສຳເລັດ (Done)"

👤 ກຳຈັດຄວາມຄຸມເຄືອ ແມ່ນການສ້າງຄວາມເຂົ້າໃຈທີ່ກົງກັນລະຫວ່າງ Product Owner ແລະ Development Team

✉ ເປັນ "ບົດທົດສອບ" ສຳລັບທີມ QA

☰ ຮູບແບບ

ຮູບແບບ "Given-When-Then"

Given [ສະພາບການເລີ່ມຕົ້ນ]
When [ການກະທຳໃດໜຶ່ງເກີດຂຶ້ນ]
Then [ຜົນລັບທີ່ຄາດຫວັງຄວນຈະເປັນ...]

💡 ຕົວຢ່າງ

📖 User Story

"ໃນຖານະສະມາຊິກຂອງສະໝຸດ, ຂ້ອຍຕ້ອງການຄົ້ນຫາປຶ້ມດ້ວຍຊື່ຜູ້ແຕ່ງ, ເພື່ອທີ່ຂ້ອຍຈະສາມາດຊອກຫາປຶ້ມທັງໝົດຂອງຜູ້ແຕ່ງທີ່ຂ້ອຍມັກໄດ້ຢ່າງໄວ."

✖ Acceptance Criteria

ສະພາບການ 1

Given ຂ້ອຍຢູ່ໜ້າຄົ້ນຫາ,
When ຂ້ອຍພິມຊື່ຜູ້ແຕ່ງທີ່ "ມີຢູ່ຈິງ" ໃນຖານຂໍ້ມູນ ແລ້ວກົດປຸ່ມຄົ້ນຫາ,
Then ລະບົບຕ້ອງສະແດງລາຍການປຶ້ມທັງໝົດທີ່ຂຽນໂດຍຜູ້ແຕ່ງຄົນນັ້ນ

ສະພາບການ 2

Given ຂ້ອຍຢູ່ໜ້າຄົ້ນຫາ,
When ຂ້ອຍພິມຊື່ຜູ້ແຕ່ງທີ່ "ບໍ່ມີຢູ່ຈິງ" ໃນຖານຂໍ້ມູນ ແລ້ວກົດປຸ່ມຄົ້ນຫາ,
Then ລະບົບຕ້ອງສະແດງຂໍ້ຄວາມວ່າ "ບໍ່ພົບຜົນການຄົ້ນຫາ"

ສະພາບການ 3

Given ຂ້ອຍຢູ່ໜ້າຄົ້ນຫາ,
When ຂ້ອຍບໍ່ໄດ້ພິມຫຍັງເລີຍໃນຊ່ອງຄົ້ນຫາ ແລ້ວກົດປຸ່ມຄົ້ນຫາ,
Then ປຸ່ມຄົ້ນຫາຄວນຈະກົດບໍ່ໄດ້ ຫຼື ລະບົບຄວນສະແດງຂໍ້ຄວາມເຕືອນໃຫ້ໃສ່ຂໍ້ມູນ

ສະຫຼຸບ



ຄວາມສໍາຄັນຂອງວິສະວະກໍາຄວາມຕ້ອງການ



ເປັນພື້ນຖານທີ່ສໍາຄັນທີ່ສຸດຂອງການພັດທະນາຊອບແວ



ການກໍານົດຄວາມຕ້ອງການທີ່ຜິດພາດ ເປັນສາເຫດອັນດັບໜຶ່ງທີ່ເຮັດໃຫ້ໂຄງການພັດທະນາຊອບແວບໍ່ສໍາເລັດ



ການແຍກປະເພດຄວາມຕ້ອງການ (Functional vs. Non-functional) ມີຜົນຕໍ່ການອອກແບບສະຖາປັດຕະຍະກໍາ

ຂະບວນການວິສະວະກໍາຄວາມຕ້ອງການ



ການລວບລວມ



ການວິເຄາະ



ການກໍານົດ



ການກວດສອບ



ມີການຈັດການເປັນແນວທາງທີ່ເຊື່ອມໂຍງກັບທຸກຂັ້ນຕອນ



ເທັກນິກ ແລະ ເຄື່ອງມື



ການສຳພາດ



ເວີກຊັອບ



ການສຳຫຼວດ



ການສັງເກດການ
(Observation)



ການວິເຄາະເອກະສານ
(Document Analysis)



ໃນໂລກຂອງ Agile, User Stories ແລະ Acceptance Criteria ແມ່ນເຄື່ອງມືທີ່ມີປະສິດທິຜົນ



ການເຂົ້າໃຈສະພາບການຕ່າງໆ ແລະ ການເລືອກໃຊ້ເທັກນິກທີ່ຖືກຕ້ອງກັບສະຖານະການ

ການເຂົ້າໃຈບັນຫາຕ່າງໆແລະການເລືອກໃຊ້ເທັກນິກ/ເຄື່ອງມືທີ່ຖືກຕ້ອງກັບສະຖານະການ ຄືສິ່ງທີ່ປຸງບອກລະຫວ່າງນັກວິເຄາະລະບົບມືໃໝ່ ແລະ ມືອາຊີບ

ຄໍາຖາມ ແລະ ການສົນທະນາ



ຂໍ້ມູນຕິດຕໍ່



ອີເມລ: Bouasoth@nuol.eud.la



ເວັບໄຊທີ່ກ່ຽວຂ້ອງ: www.fns.nuol.edu.la



ຄໍາແນະນໍາ



ວິສະວະກໍາຄວາມຕ້ອງການ (Requirement Engineering)

ໂດຍຊື່ຜູ້ຂຽນ, ປີ 2023



ການວິເຄາະລະບົບແລະການອອກແບບຊອບແວ
ໂດຍ Ian Sommerville, ສະບັບທີ 10



ອ່ານເພີ່ມເຕີມ



Agile Software Development, Scrum, and
Extreme Programming

ໂດຍ Robert C. Martin



User Stories Applied: For Agile Software
Development

ໂດຍ Mike Cohn

ຂອບໃຈທີ່ເຂົ້າຮ່ວມຮັບຟັງໃນການສອນນີ້!

ກະລຸນາບອກຂໍ້ມູນເພີ່ມເຕີມຫຼືຄໍາຖາມທີ່ທ່ານຕ້ອງການ